

```

1 #!/usr/bin/env python
2 """
3 Title: Retrieval Augmented Generation for Biomedical Database with AI Language
4       Model
5 Description: This script combines three approaches for Retrieval-Augmented
6              Generation (RAG)
7              using multiple AI models (MedCPT-based encoders, llama_index with
8              OpenAI embeddings,
9              and direct LLM query via llama_index). It processes PDFs, builds
10             FAISS indices,
11             performs dense retrieval, re-ranking, LLM answer generation, and
12             evaluates results
13             using ROUGE, BLEU, and RAGAS metrics.
14 Author: Vatsal P. Patel
15 Date: 12/02/25
16 """
17
18 #####
19 # 0. Package Imports and Environment Setup
20 #####
21 import os
22 import shutil
23 import json
24 import csv
25 import numpy as np
26 import torch
27 import faiss
28 import PyPDF2
29 from dotenv import load_dotenv
30
31 # Transformers & Tokenizers
32 from transformers import AutoTokenizer, AutoModel,
33                        AutoModelForSequenceClassification
34
35 # Llama-index / LangChain related imports
36 from llama_index.core import VectorStoreIndex, SimpleDirectoryReader,
37                        StorageContext, load_index_from_storage
38 from llama_index.embeddings.openai import OpenAIEmbedding
39 from llama_index.llms.openai import OpenAI as LlamaOpenAI
40 from llama_index.core.schema import TextNode
41 from llama_index.core.ingestion import IngestionPipeline
42 from llama_index.core.node_parser import SentenceSplitter
43 from langchain.vectorstores import FAISS as LC_FAISS
44
45 # Evaluation libraries
46 from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction
47 from rouge_score import rouge_scorer
48
49 # RAGAS evaluation
50 from datasets import Dataset
51 from ragas import evaluate
52 from ragas.metrics import faithfulness, answer_correctness, context_recall,

```

```

context_precision, answer_relevancy

46
47 # Load environment variables from .env file if available
48 load_dotenv()
49
50 # Set your OpenAI API key (replace with your actual key)
51 OPENAI_API_KEY = os.getenv("OPENAI_API_KEY", "YOUR_OPENAI_API_KEY_HERE")
52 os.environ["OPENAI_API_KEY"] = OPENAI_API_KEY
53
54 # Global configuration parameters (adjust as needed)
55 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
56
57 #####
58 # 1. Common Utility Functions
59 #####
60 def extract_text_from_pdf(pdf_path):
61     """Extract text from a PDF file."""
62     text = ""
63     with open(pdf_path, 'rb') as f:
64         reader = PyPDF2.PdfReader(f)
65         for page in reader.pages:
66             page_text = page.extract_text()
67             if page_text:
68                 text += page_text + "\n"
69     return text
70
71 def chunk_text(text, chunk_size=500, overlap=50):
72     """Chunk text into overlapping segments."""
73     words = text.split()
74     chunks = []
75     start = 0
76     while start < len(words):
77         end = start + chunk_size
78         chunk = words[start:end]
79         if not chunk:
80             break
81         chunks.append(" ".join(chunk))
82         start += (chunk_size - overlap)
83     return chunks
84
85 #####
86 # 2. Method 1: RAG using MedCPT Encoders & FAISS
87 #####
88 def method1_pipeline(pdf_folder, output_dir):
89     """
90     Pipeline using MedCPT-based article/query/cross encoders for:
91     - PDF text extraction and chunking
92     - Document embedding and FAISS index building
93     - Dense retrieval, cross-encoder re-ranking, and LLM answer generation
94     - Evaluation (ROUGE, BLEU, RAGAS)
95     """
96     # Configuration parameters for MedCPT models
97     INDEX_SAVE_PATH = os.path.join(output_dir, "faiss_index_medcpt.bin")

```

```

98 DOC_META_PATH = os.path.join(output_dir, "doc_metadata_medcpt.json")
99 MAX_ARTICLE_LENGTH = 512
100 MAX_QUERY_LENGTH = 64
101 CHUNK_SIZE = 500
102 OVERLAP = 50
103
104 # Model names (ensure these are accessible)
105 ARTICLE_ENCODER_MODEL = "ncbi/MedCPT-Article-Encoder"
106 QUERY_ENCODER_MODEL = "ncbi/MedCPT-Query-Encoder"
107 CROSS_ENCODER_MODEL = "ncbi/MedCPT-Cross-Encoder"
108
109 #####
110 # Step 2: Process PDFs and Prepare Data
111 #####
112 doc_metadata = []
113 doc_text_chunks = []
114 doc_id = 0
115
116 for filename in os.listdir(pdf_folder):
117     if filename.lower().endswith(".pdf"):
118         pdf_path = os.path.join(pdf_folder, filename)
119         text = extract_text_from_pdf(pdf_path)
120         chunks = chunk_text(text, chunk_size=CHUNK_SIZE, overlap=OVERLAP)
121         for chunk_id, chunk in enumerate(chunks):
122             doc_metadata.append({
123                 "doc_id": doc_id,
124                 "filename": filename,
125                 "chunk_id": chunk_id,
126                 "text": chunk
127             })
128             doc_id += 1
129             doc_text_chunks.extend(chunks)
130
131 #####
132 # Step 3: Embedding Documents (Article Encoder)
133 #####
134 article_tokenizer = AutoTokenizer.from_pretrained(ARTICLE_ENCODER_MODEL)
135 article_model = AutoModel.from_pretrained(ARTICLE_ENCODER_MODEL).to(DEVICE)
136 article_model.eval()
137
138 def embed_documents(doc_chunks, batch_size=8):
139     all_embeds = []
140     for i in range(0, len(doc_chunks), batch_size):
141         batch = doc_chunks[i:i+batch_size]
142         with torch.no_grad():
143             encoded = article_tokenizer(batch, truncation=True, padding=
144                                     True, return_tensors='pt', max_length=MAX_ARTICLE_LENGTH)
145             for k in encoded:
146                 encoded[k] = encoded[k].to(DEVICE)
147             outputs = article_model(**encoded).last_hidden_state[:, 0, :]
148             all_embeds.append(outputs.cpu().numpy())
149     return np.vstack(all_embeds) if all_embeds else np.array([])

```

```

149 all_embeddings = embed_documents(doc_text_chunks)
150 print("Method1: Total embeddings shape:", all_embeddings.shape)
151
152
153 #####
154 # Step 4: Build FAISS Index
155 #####
156 dimension = all_embeddings.shape[1]
157 index = faiss.IndexFlatIP(dimension) # Using inner product similarity
158 index.add(all_embeddings)
159 faiss.write_index(index, INDEX_SAVE_PATH)
160 with open(DOC_META_PATH, 'w') as f:
161     json.dump(doc_metadata, f)
162 print("Method1: FAISS index built and metadata saved.")
163
164 #####
165 # Step 5: Query & Re-Ranking Functions
166 #####
167 query_tokenizer = AutoTokenizer.from_pretrained(QUERY_ENCODER_MODEL)
168 query_model = AutoModel.from_pretrained(QUERY_ENCODER_MODEL).to(DEVICE)
169 query_model.eval()
170
171 def embed_query(queries):
172     with torch.no_grad():
173         encoded = query_tokenizer(queries, truncation=True, padding=True,
174                                   return_tensors='pt', max_length=MAX_QUERY_LENGTH)
175         for k in encoded:
176             encoded[k] = encoded[k].to(DEVICE)
177         outputs = query_model(**encoded).last_hidden_state[:, 0, :]
178         return outputs.cpu().numpy()
179
180 cross_tokenizer = AutoTokenizer.from_pretrained(CROSS_ENCODER_MODEL)
181 cross_model = AutoModelForSequenceClassification.from_pretrained(
182     CROSS_ENCODER_MODEL).to(DEVICE)
183 cross_model.eval()
184
185 def rerank(query, candidates):
186     pairs = [[query, candidate["text"]] for candidate in candidates]
187     with torch.no_grad():
188         encoded = cross_tokenizer(pairs, truncation=True, padding=True,
189                                   return_tensors="pt", max_length=512)
190         for k in encoded:
191             encoded[k] = encoded[k].to(DEVICE)
192         logits = cross_model(**encoded).logits.squeeze(dim=1)
193     return logits.cpu().numpy()
194
195 def search_with_rerank(query, k=5):
196     query_embedding = embed_query([query])
197     scores, inds = index.search(query_embedding, k)
198     candidates = []
199     for score, ind in zip(scores[0], inds[0]):
200         entry = doc_metadata[ind]
201         entry["retrieval_score"] = float(score)

```

```

199         candidates.append(entry)
200     rerank_scores = rerank(query, candidates)
201     for i, score in enumerate(rerank_scores):
202         candidates[i]["rerank_score"] = float(score)
203     candidates = sorted(candidates, key=lambda x: x["rerank_score"],
204                          reverse=True)
205     return candidates
206
207 #####
208 # Step 6: LLM-based Answer Generation (Using OpenAI via llama_index
209 # wrapper)
210 #####
211 def get_llm_answer(query, retrieved_candidates):
212     context_text = " ".join([cand["text"] for cand in retrieved_candidates
213                               ])
214     prompt = f"""
215 You are a knowledgeable assistant. Use the context below to answer the
216 question in one word or few.
217
218 Context:
219 {context_text}
220
221 Question: {query}
222
223 Answer (in one word or few):
224 """
225     llm = LlamaOpenAI(model="gpt-4", temperature=0)
226     response = llm.complete(prompt)
227     return response, context_text
228
229 #####
230 # Step 7: Evaluation using ROUGE & BLEU
231 #####
232 sample_queries = [
233     "Which biomarker is significantly elevated in the plasma of Gaucher
234     Disease Type 1 patients?",
235     "What therapy reduces urinary GlcSph levels in Gaucher Disease
236     patients?"
237     # ... add more queries as needed
238 ]
239 ground_truths = [
240     "Glucosylsphingosine (GlcSph).",
241     "Enzyme Replacement Therapy (ERT).",
242     # ... corresponding ground truths
243 ]
244
245 def evaluate_results_with_rerank(queries, ground_truths, k=3):
246     rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2',
247                                                         'rougeL'], use_stemmer=True)
248     smoothing_function = SmoothingFunction().method4
249     results = []
250     for query, ground_truth in zip(queries, ground_truths):
251         retrieved_results = search_with_rerank(query, k=k)

```

```

245         llm_answer, context_text = get_llm_answer(query, retrieved_results
246         )
247         response_text = str(llm_answer).strip()
248         rouge_scores = rouge_scorer_instance.score(ground_truth,
249         response_text)
250         bleu_score = sentence_bleu(
251         [ground_truth.split()],
252         response_text.split(),
253         smoothing_function=smoothing_function
254         )
255         results.append({
256             "query": query,
257             "ground_truth": ground_truth,
258             "response": response_text,
259             "retrieved_context": context_text,
260             "rouge1": rouge_scores['rouge1'].fmeasure,
261             "rouge2": rouge_scores['rouge2'].fmeasure,
262             "rougeL": rouge_scores['rougeL'].fmeasure,
263             "bleu": bleu_score
264         })
265     return results
266
267 eval_results = evaluate_results_with_rerank(sample_queries, ground_truths)
268 csv_path = os.path.join(output_dir, "evaluation_metrics_medcpt.csv")
269 with open(csv_path, mode='w', newline='', encoding='utf-8') as csvfile:
270     writer = csv.writer(csvfile, quoting=csv.QUOTE_MINIMAL, escapechar='\\
271     ')
272     writer.writerow(["Query", "Ground Truth", "Response", "Retrieved
273     Context", "ROUGE-1", "ROUGE-2", "ROUGE-L", "BLEU"])
274     for result in eval_results:
275         writer.writerow([
276             result["query"],
277             result["ground_truth"],
278             result["response"],
279             result["retrieved_context"],
280             f"{result['rouge1']:.4f}",
281             f"{result['rouge2']:.4f}",
282             f"{result['rougeL']:.4f}",
283             f"{result['bleu']:.4f}"
284         ])
285     print(f"Method1: Evaluation metrics saved to {csv_path}")
286
287 #####
288 # Step 8: RAGAS Evaluation (Qualitative)
289 #####
290 data_samples = {
291     'question': [result["query"] for result in eval_results],
292     'answer': [result["response"] for result in eval_results],
293     'contexts': [[result["retrieved_context"]] for result in eval_results
294     ],
295     'ground_truth': [result["ground_truth"] for result in eval_results]
296 }
297 dataset = Dataset.from_dict(data_samples)

```

```

293     score = evaluate(dataset, metrics=[faithfulness, answer_correctness,
294                             context_recall, context_precision, answer_relevancy])
295     df = score.to_pandas()
296     ragas_csv = os.path.join(output_dir, "ragas_evaluation_medcpt.csv")
297     df.to_csv(ragas_csv, index=False)
298     print(f"Method1: RAGAS evaluation scores saved to {ragas_csv}")
299
300 #####
301 # 3. Method 2: RAG using llama_index (OpenAI Embeddings + FAISS)
302 #####
303 def method2_pipeline(pdf_folder, output_dir):
304     """
305     Pipeline using llama_index to:
306     - Load and chunk PDFs
307     - Build a FAISS index using OpenAI embeddings
308     - Retrieve documents and answer queries via OpenAI LLM
309     - Evaluate responses using ROUGE, BLEU, and RAGAS metrics
310     """
311     # Configuration parameters
312     PERSIST_DIR = os.path.join(output_dir, "storage_llama")
313     DOC_META_PATH = os.path.join(output_dir, "doc_metadata_llama.json")
314     INDEX_SAVE_PATH = os.path.join(output_dir, "faiss_index_llama.bin")
315     EMBEDDING_MODEL = "text-embedding-ada-002"
316     LLM_MODEL = "gpt-4"
317     CHUNK_SIZE = 500
318     OVERLAP = 50
319
320     # Step 1: Read PDFs, chunk text, and create nodes
321     documents = []
322     doc_metadata = []
323     doc_id = 0
324     for filename in os.listdir(pdf_folder):
325         if filename.lower().endswith(".pdf"):
326             pdf_path = os.path.join(pdf_folder, filename)
327             text = extract_text_from_pdf(pdf_path)
328             splitter = SentenceSplitter(chunk_size=CHUNK_SIZE, chunk_overlap=
329                                     OVERLAP)
330             chunks = splitter.split_text(text)
331             for chunk_id, chunk in enumerate(chunks):
332                 node = TextNode(text=chunk, id_=f"{doc_id}_{chunk_id}")
333                 doc_metadata.append({
334                     "doc_id": doc_id,
335                     "filename": filename,
336                     "chunk_id": chunk_id,
337                     "text": chunk
338                 })
339                 documents.append(node)
340             doc_id += 1
341
342     # Step 2: Build FAISS index with OpenAI embeddings
343     storage_context = StorageContext.from_defaults()
344     embedding_model = OpenAIEmbedding(model=EMBEDDING_MODEL)
345     dimension = 1536 # Dimension for text-embedding-ada-002

```

```

344 faiss_index = faiss.IndexFlatL2(dimension)
345 # Compute embeddings for each document node
346 for node in documents:
347     embed = embedding_model.get_text_embedding(node.text)
348     faiss_index.add(np.array(embed).reshape(1, -1))
349 # Save metadata and index
350 with open(DOC_META_PATH, 'w') as f:
351     json.dump(doc_metadata, f)
352 faiss.write_index(faiss_index, INDEX_SAVE_PATH)
353 print("Method2: FAISS index built using llama_index pipeline.")
354
355 # Step 3: Query Pipeline using llama_index LLM
356 def retrieve_documents(query, top_k=3):
357     query_embedding = embedding_model.get_text_embedding(query)
358     scores, indices = faiss_index.search(np.array(query_embedding).reshape(
359         (1, -1), top_k)
360     results = []
361     for i, score in zip(indices[0], scores[0]):
362         if i != -1:
363             results.append({
364                 "score": score,
365                 "content": doc_metadata[i]["text"],
366                 "metadata": doc_metadata[i]
367             })
368     return results
369
370 def query_with_llm(query, retrieved_docs):
371     context = "\n".join([doc["content"] for doc in retrieved_docs])
372     prompt = f"""
373 You are a knowledgeable assistant. Use the context below to answer the
374 question concisely in as few words as possible.
375 Context:
376 {context}
377 Question: {query}
378 Answer (in minimal words):"""
379     llm = LlamaOpenAI(model=LLM_MODEL, temperature=0)
380     return llm.complete(prompt)
381
382 # Step 4: Evaluation using ROUGE and BLEU
383 sample_queries = [
384     "What metabolites are associated with breast cancer?"
385     # Add additional queries as needed
386 ]
387 ground_truths = [
388     "Sample ground truth answer for breast cancer metabolites."
389     # Add corresponding ground truths
390 ]
391 rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2', '
392     rougeL'], use_stemmer=True)
393 smoothing_function = SmoothingFunction().method4

```



```

394 def evaluate_results_with_llm(queries, ground_truths, top_k=3):
395     results = []
396     for query, ground_truth in zip(queries, ground_truths):
397         retrieved_docs = retrieve_documents(query, top_k=top_k)
398         context_text = "\n".join([doc["content"] for doc in retrieved_docs
399                                   ])
400         response = query_with_llm(query, retrieved_docs)
401         response_text = str(response).strip()
402         rouge_scores = rouge_scorer_instance.score(ground_truth,
403             response_text)
404         bleu_score = sentence_bleu(
405             [ground_truth.split()],
406             response_text.split(),
407             smoothing_function=smoothing_function,
408             weights=(0.5, 0.5, 0, 0)
409         )
410         results.append({
411             "query": query,
412             "ground_truth": ground_truth,
413             "response": response_text,
414             "retrieved_context": context_text,
415             "rouge1": rouge_scores['rouge1'].fmeasure,
416             "rouge2": rouge_scores['rouge2'].fmeasure,
417             "rougeL": rouge_scores['rougeL'].fmeasure,
418             "bleu": bleu_score
419         })
420     return results
421
422 eval_results = evaluate_results_with_llm(sample_queries, ground_truths)
423 csv_path = os.path.join(output_dir, "evaluation_metrics_llama.csv")
424 with open(csv_path, mode='w', newline='', encoding='utf-8') as csvfile:
425     writer = csv.writer(csvfile)
426     writer.writerow(["Query", "Ground Truth", "Response", "Retrieved
427                     Context", "ROUGE-1", "ROUGE-2", "ROUGE-L", "BLEU"])
428     for result in eval_results:
429         writer.writerow([
430             result["query"],
431             result["ground_truth"],
432             result["response"],
433             result["retrieved_context"],
434             f"{result['rouge1']:.4f}",
435             f"{result['rouge2']:.4f}",
436             f"{result['rougeL']:.4f}",
437             f"{result['bleu']:.4f}"
438         ])
439     print(f"Method2: Evaluation metrics saved to {csv_path}")
440
441 # Step 5: RAGAS Evaluation for Method2
442 data_samples = {
443     'question': [res["query"] for res in eval_results],
444     'answer': [res["response"] for res in eval_results],
445     'contexts': [[res["retrieved_context"]] for res in eval_results],
446     'ground_truth': [res["ground_truth"] for res in eval_results]
447 }

```

```

444 }
445 dataset = Dataset.from_dict(data_samples)
446 score = evaluate(dataset, metrics=[faithfulness, answer_correctness,
447     context_recall, context_precision, answer_relevancy])
447 df = score.to_pandas()
448 ragas_csv = os.path.join(output_dir, "ragas_evaluation_llama.csv")
449 df.to_csv(ragas_csv, index=False)
450 print(f"Method2: RAGAS evaluation scores saved to {ragas_csv}")
451
452 #####
453 # 4. Method 3: Direct LLM Query with llama_index (Concise Responses)
454 #####
455 def method3_pipeline(output_dir):
456     """
457     Pipeline that directly queries OpenAI LLM (via llama_index wrapper) for
458     concise, one-word answers.
459     It then evaluates the responses with ROUGE and BLEU, and performs a RAGAS
460     evaluation without retrieval context.
461     """
462     LLM_MODEL = "gpt-4"
463     TEMPERATURE = 0
464
465     def test_llama_index_openai_llm(prompt, model=LLM_MODEL, temperature=
466         TEMPERATURE):
467         concise_prompt = f"{prompt}\nAnswer in one word or less:"
468         llm = LlamaOpenAI(model=model, temperature=temperature)
469         try:
470             response = llm.complete(concise_prompt)
471             return response
472         except Exception as e:
473             return f"Error: {e}"
474
475     # Sample queries and ground truths for evaluation
476     sample_queries = [
477         "What is the capital of France?",
478         "What is the chemical symbol for water?"
479     ]
480     ground_truths = [
481         "Paris",
482         "H2O"
483     ]
484     rouge_scorer_instance = rouge_scorer.RougeScorer(['rouge1', 'rouge2', '
485         rougeL'], use_stemmer=True)
486     smoothing_function = SmoothingFunction().method4
487
488     def evaluate_responses(queries, ground_truths):
489         results = []
490         for query, ground_truth in zip(queries, ground_truths):
491             response = test_llama_index_openai_llm(query)
492             response_text = response.text.strip() if hasattr(response, "text")
493                 else str(response).strip()
494             rouge_scores = rouge_scorer_instance.score(ground_truth,
495                 response_text)

```

```

490     bleu_score = sentence_bleu(
491         [ground_truth.split()],
492         response_text.split(),
493         smoothing_function=smoothing_function
494     )
495     results.append({
496         "query": query,
497         "ground_truth": ground_truth,
498         "response": response_text,
499         "rouge1": rouge_scores['rouge1'].fmeasure,
500         "rouge2": rouge_scores['rouge2'].fmeasure,
501         "rougeL": rouge_scores['rougeL'].fmeasure,
502         "bleu": bleu_score
503     })
504     return results
505
506 eval_results = evaluate_responses(sample_queries, ground_truths)
507 csv_path = os.path.join(output_dir, "evaluation_metrics_direct_llm.csv")
508 with open(csv_path, mode='w', newline='', encoding='utf-8') as csvfile:
509     writer = csv.writer(csvfile)
510     writer.writerow(["Query", "Ground Truth", "Response", "ROUGE-1", "
511         ROUGE-2", "ROUGE-L", "BLEU"])
512     for result in eval_results:
513         writer.writerow([
514             result["query"],
515             result["ground_truth"],
516             result["response"],
517             f"{result['rouge1']:.4f}",
518             f"{result['rouge2']:.4f}",
519             f"{result['rougeL']:.4f}",
520             f"{result['bleu']:.4f}"
521         ])
522     print(f"Method3: Evaluation metrics saved to {csv_path}")
523
524 # RAGAS Evaluation (without retrieval context)
525 data_samples = {
526     'question': [res["query"] for res in eval_results],
527     'answer': [res["response"] for res in eval_results],
528     'contexts': [[] for _ in eval_results],
529     'ground_truth': [res["ground_truth"] for res in eval_results]
530 }
531 dataset = Dataset.from_dict(data_samples)
532 score = evaluate(dataset, metrics=[faithfulness, answer_correctness,
533     context_recall, context_precision, answer_relevancy])
534 df = score.to_pandas()
535 ragas_csv = os.path.join(output_dir, "ragas_evaluation_direct_llm.csv")
536 df.to_csv(ragas_csv, index=False)
537 print(f"Method3: RAGAS evaluation scores saved to {ragas_csv}")
538
539 #####
540 # 5. Main: Run All Pipelines
541 #####
542 if __name__ == "__main__":

```

```
541 # Define paths      adjust these as needed.
542 BASE_DIR = os.getcwd()
543 PDF_FOLDER = os.path.join(BASE_DIR, "sample_pdf_rag") # Folder containing
544               your PDFs
545 OUTPUT_DIR = os.path.join(BASE_DIR, "rag_output")
546 os.makedirs(OUTPUT_DIR, exist_ok=True)
547
548 print("Starting Method 1: MedCPT-based RAG pipeline...")
549 method1_pipeline(PDF_FOLDER, OUTPUT_DIR)
550
551 print("\nStarting Method 2: llama_index-based RAG pipeline...")
552 method2_pipeline(PDF_FOLDER, OUTPUT_DIR)
553
554 print("\nStarting Method 3: Direct LLM Query pipeline...")
555 method3_pipeline(OUTPUT_DIR)
556
557 print("\nAll pipelines completed. Check the output directory for CSV
558       evaluation files.")
```